

Project Proposal

AI Architecture Decision Intelligence Platform (ADIP)

1. Introduction

Software architecture is one of the most important parts of software development. Decisions such as choosing a monolithic architecture or microservices, selecting PostgreSQL or MongoDB, deciding whether caching is required, and selecting a deployment strategy directly affect the performance, scalability, maintenance effort, and operational cost of a software system. These decisions are usually taken during the early stages of development, when very little practical information is available. In many cases, developers rely on previous experience, online articles, or trial-and-error. If the chosen architecture does not suit the project requirements, changing it later becomes expensive and time-consuming.

2. Problem Statement

Although many tools are available for creating architecture diagrams and documentation, they do not help developers predict the consequences of their architectural decisions. Startups and student developers often struggle to decide which architecture is most suitable for their application. A wrong decision can lead to poor response time, unnecessary cloud expenditure, limited scalability, and difficult maintenance. There is a need for a system that can analyze project requirements and provide practical recommendations before development starts.

3. Proposed Solution

The proposed project aims to develop an AI-based Architecture Decision Intelligence Platform that assists developers in selecting a suitable software architecture. The user will provide project details such as expected users, requests per second, budget, data type, latency requirements, and team size. Based on these inputs, the platform will generate multiple architecture options, estimate their performance and infrastructure cost, compare them, and recommend the most appropriate solution.

Machine learning models will be used to estimate important software quality metrics such as response time, scalability, reliability, and monthly infrastructure cost. After generating the recommendation, the platform will use the Llama Large Language Model through LangChain to explain the reasoning behind the decision in simple language. Instead of giving only a final answer, the system will describe the advantages, disadvantages, and trade-offs of each architecture.

4. Objectives

- To study the requirements of a software project before development begins.
- To generate multiple architecture alternatives based on project constraints.
- To predict latency, scalability, reliability, and cloud cost using machine learning models.
- To compare architecture options using a recommendation engine.
- To integrate the Llama Large Language Model through LangChain for explainable recommendations.
- To help developers make informed architecture decisions.

5. Technology Stack

Component	Technology
Frontend	Next.js (React), Tailwind CSS
Backend	Node.js, Express.js
Machine Learning	Python, Scikit-learn, XGBoost
LLM	Llama
LLM Framework	LangChain
Database	PostgreSQL, Redis
Deployment	Docker, AWS

6. Expected Outcome

The completed system will work as a decision-support platform for software architects and developers. It will help users compare different architecture choices before implementation, estimate their expected performance and operational cost, and understand the reasoning behind every recommendation. This can reduce design mistakes, improve software quality, and support better planning for future growth.

7. Future Scope

In the future, the platform can be extended to support additional cloud providers, integrate live cloud pricing APIs, recommend migration strategies, analyze security and compliance requirements, and continuously improve its recommendations using real-world deployment data.